# Python语言

清華大學出版社
TSINGHUA UNIVERSITY PRESS

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.5 小结

习题

# 3.1 条件语句



流程控制是指在程序运行时，对指令运行顺序的控制。

通常，程序流程结构分为三种：顺序结构、分支结构和循环结构。顺序结构是程序中最常见的流程结构，按照程序中语句的先后顺序，自上而下依次执行，称为顺序结构；分支结构则根据if条件的真假（True或者False）来决定要执行的代码；循环结构则是重复执行相同的代码，直到整个循环完成或者使用break强制跳出循环。

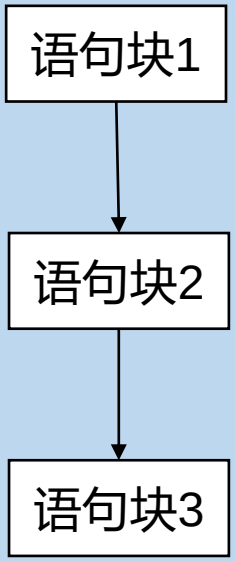
Python语言中，一般来说，我们使用if语句实现分支结构，用for和while语句实现循环结构。

流程图，是使用图形来表示流程控制的一种方法，是一种传统的算法表示方法，用特定的图形符号和文字对流程和算法加以说明，叫做算法的图，也称为流程图。俗话说千言万语不如一张图。

流程图有它自己的规范，按照这样的规范所画出的流程图，便于技术人员之间的交流，也是软件项目开发所必备的基本组成部分，因此画流程图也应是开发者的基本功。

| 符号 | 说明 |
| --- | --- |
| | 圆角矩形用来表示“开始”与“结束”。 |
| | 矩形用来表示要执行的动作或算法。 |
| | 菱形用来表示问题判断。 |
| | 平行四边形用来表示输入输出。 |
| | 箭头用来代表工作流方向。 |

顺序结构是程序中最常见的流程结构，按照程序中语句的先后顺序，自上而下依次执行，称为顺序结构；




```
1  a=2
2  b=3
3  print(a+b)
```

Run: 顺序结构

```
D:\Python\Python36\python.exe F:/py
5

Process finished with exit code 0
```

# 3.1 条件语句



条件语句是用来判断给定的条件是否满足，并根据判断的结果（True或False）决定是否执行或如何执行后续流……程的语句，它使代码的执行顺序有了更多选择，以实现更多的功能。

一般来说，条件表达式是由条件运算符和相应的数据所构成的，在Python中，所有合法的表达式都可以作为条件表达式。条件表达式的值只要不是False、0、空值（None）、空列表、空集合、空元组、空字符串等，其它均为True。

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.5 小结

习题

f语句是由if发起的一个条件语句，在满足此条件后执行相应内容，Python的语句基本结构如下。

```
if 表达式1:
    语句块1
elif 表达式2:
    语句块2
……
else:
    语句块n
```

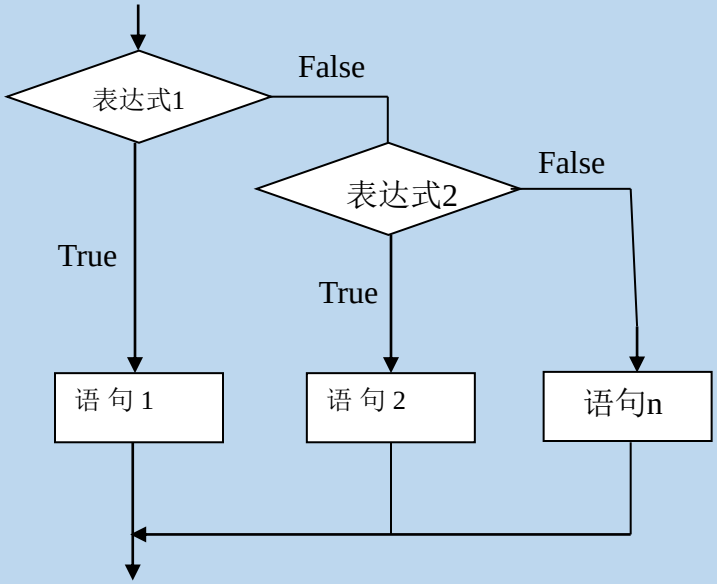



图3.1分支选择结构

这里的elif，为else if 的缩写，同时需要注意的是：

1、else、elif为if语句的子语句块，不能独立使用。

2、每个条件后面要使用冒号“:”，表示满足条件后需要执行的语句块，后面几种其它形式的选择结构和循环结构中是冒号也是必须要有的。

3、使用缩进来划分语句块，相同缩进数的语句组成一个语句块。

4、在Python中没有switch…case语句。

单向分支选择结构是最简单的一种形式，不包含elif和else，当表达式值为True时，执行语句块，否则该语句块不执行，继续执行后面的代码。其语法如下。

if 表达式:

语句块

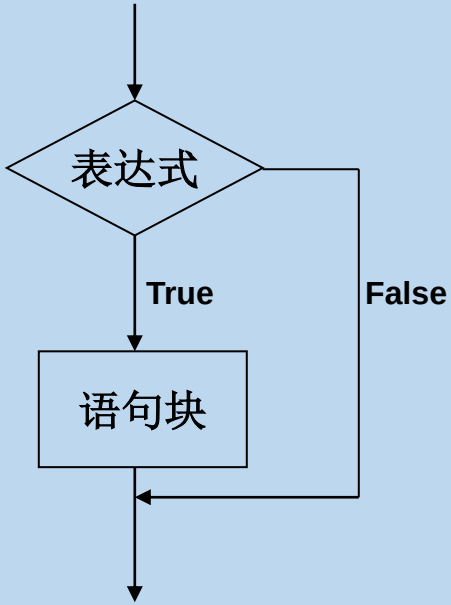



图3.2单分支选择结构

```
#判断变量a的值是否为True，是则执行a=0，否则直接输出a的值。
a = 1
if a:      # 等价于a>0或a!=0
    a = 0
print(a)   # 如果if条件的值为False则输出结果为1
```

if a

Run: demo03_02_01

```
D:\Python\Python36\python.exe F:/python/new/demo03_02_01.py
0

Process finished with exit code 0
```

双分支语句是由if和else两部分组成，当表达式的值为True时，执行语句块1否则执行语句块2。双分支选择结构的语法如下。

```
if 表达式:
    语句块1
else:
    语句块2
```

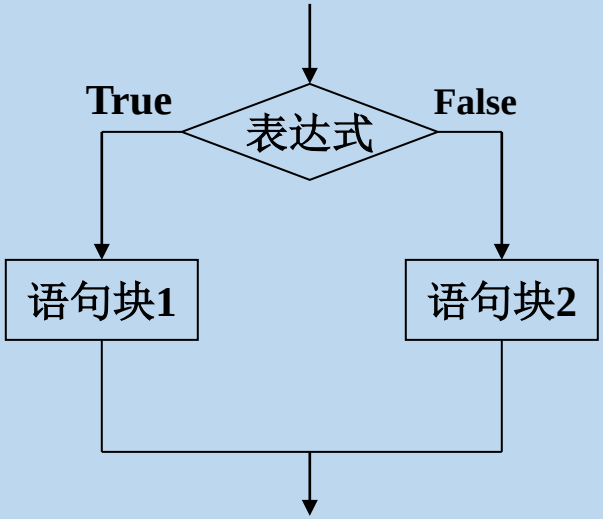



图3.3双分支选择结构

例如判断条件表达式的值是否为True，是则执行语句块1，否则执行else部分，最后再输出a的值。

```
#判断a的值是否为True，是则执行语句块1，否则执行else部分，最后再输出a的值。
a = 5
if a < 0:
    a = 10
    print(a+1)      # 如果if的条件为True，就输出11
else:
    a = 15
    print(a+2)      # 如果if的条件为False，就输出17
print(a)
```

if a < 0

Run: demo03_02_02

```
D:\Python\Python36\python.exe F:/python/new/demo03_02_02.py
17
15

Process finished with exit code 0
```

多分支选择结构由if、一个或多个elif和一个else子块组成，else子块可省略。一个if语句可以包含多个elif语句，但结尾最多只能有一个else。多分支选择结构的语法如下。

```
if  表达式1:
    语句块1
elif  表达式2:
    语句块2
elif  表达式3:
    语句块3
……
else:
    语句块n
```

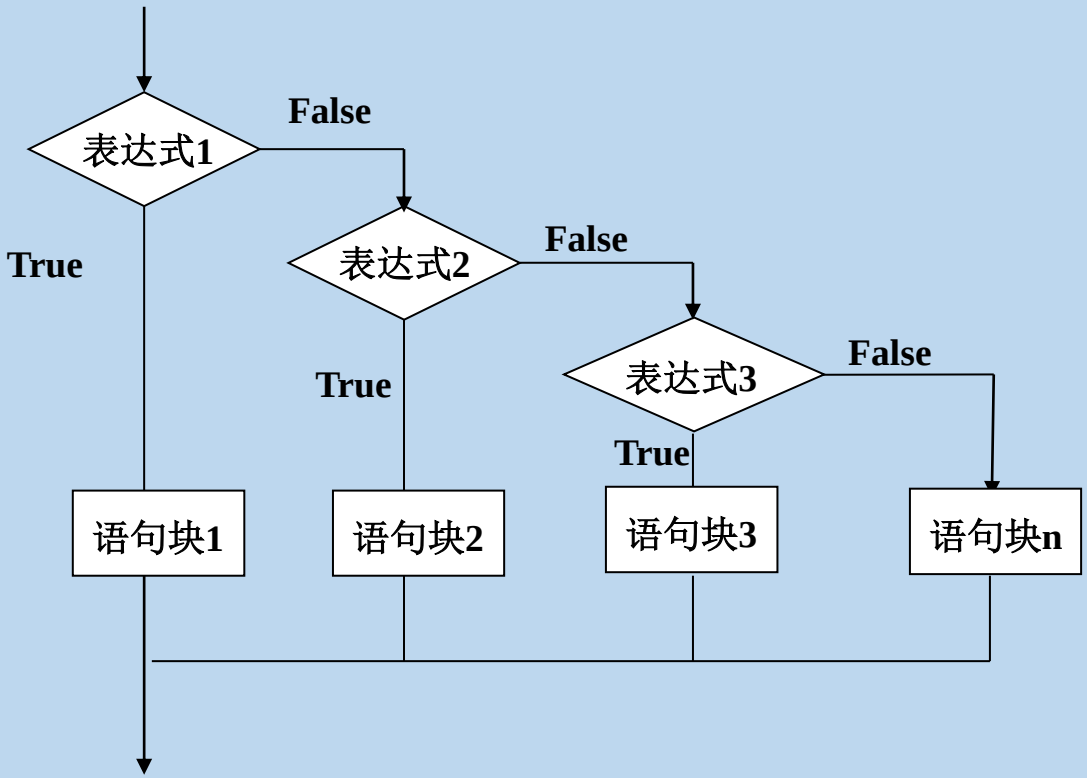



图3.4多分支选择结构

例如请输入一个正整数，判断它是否能同时被2和3整除。

```
1   # 根据你身上带的钱，来决定你今天中午能吃什么。
2   money = float(input("请输入你带的钱："))
3   if (money >= 1) and (money <= 5):     # 判断money是否在1到5之间
4       print("你可以吃包子")
5   elif (money > 5) and (money <= 10):   # 判断money是否在6到10之间
6       print("你可以吃面条")
7   elif money < 0:                       # 如果money小于0，就说明你没有钱，否则就说明你的钱大于10元
8       print("你的钱不够")
9   else:
10      print("你可以吃大餐")
```

elif (money > 5) and (money <= ...

Run: demo03_02_03

```
D:\Python\Python36\python.exe F:/python/new/demo03_02_03.py
请输入你带的钱：50
你可以吃大餐

Process finished with exit code 0
```

选择结构可以进行嵌套来表达更复杂的逻辑关系。使用选择结构嵌套时，一定要控制好不同级别的代码块的缩进，否则就不能被Python正确理解和执行。在if 语句嵌套中， if、if...else、if...elif...else它们可以进行一次或多次相互嵌套，例如结构如下。

```
if 表达式1:
    语句块1
if 表达式2:
    语句块2
else:
    if 表达式3:
        语句块3
    else:
        语句块4
```

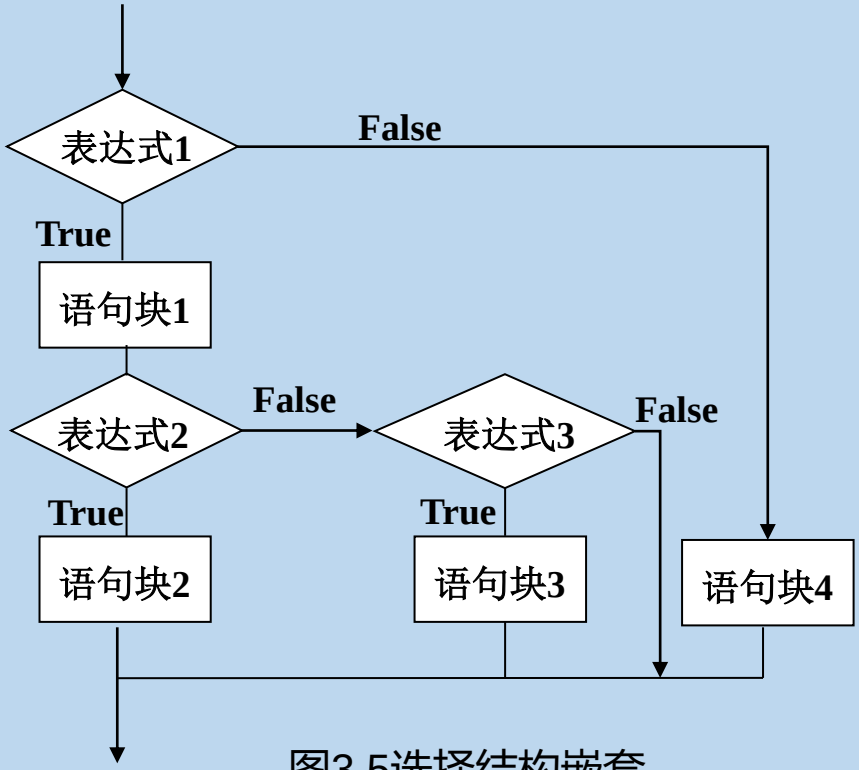



图3.5选择结构嵌套

例如请输入一个正整数，判断它是否能同时被2和3整除。

```
1  # 请输入一个正整数，判断它是否能同时被2和3整除。
2  a = int(input("请输入一个正整数："))
3  if a % 2 == 0:                # 判断一个数是否能被2整除
4      if a % 3 == 0:            # 判断一个数是否能被3整除
5          print(a)
6      else:
7          print("此数能够被2整除，但是不能被3整除！")
8  else:
9      print("此数不能被2整除！")
```

if a % 2 == 0 › if a % 3 == 0

Run: demo03_02_04

```
D:\Python\Python36\python.exe F:/python/new/demo03_02_04.py
请输入一个正整数：666
666

Process finished with exit code 0
```

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.5 小结

习题

循环，是我们生活中常见的，比如每天都要吃饭、上课、睡觉等，这就是典型的循环。循环结构是指在程序中需要反复执行某个功能而设置的一种程序结构。

Python提供for和while两种循环语句。for语句，用来遍历序列对象内的元素，通常用在已知的循环次数；while语句，提供了编写通用循环的方法。

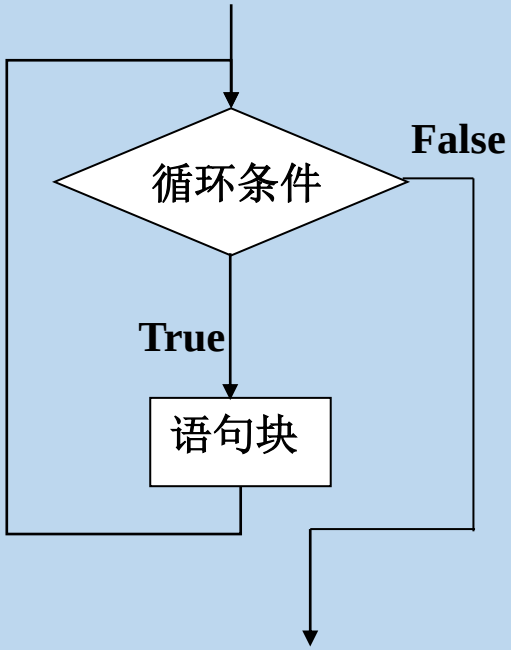



图3.6循环流程图

# 3.3 循环流程控制



for循环的语法结构跟前面讲的if...else有点类似，记的时候不要记混淆了。for执行时，依次将可迭代对象中的值赋给变量，变量每赋值一次，则执行一次循环体。循环执行结束时，如果有else部分，则执行对应的语句块。else只有在循环正常结束时执行。如果使用break跳出循环，则不会执行else部分，且根据实际编程需求，else部分可以省略。其结构如下。

```
for  变量 in 序列或迭代对象:
    循环体（语句块1）
else:
    语句块2
```

注意：

for和else后面冒号不能丢，循环体、语句块缩进严格对齐。

例如求1~100的累加和，range()函数是生成1到100的整数，Sum是累加的和。

```
1  Sum = 0
2  for s in range(1, 101):   # 循环从1到100，当101时就退出循环
3      Sum += s              # 求1到100的累加和
4  print(Sum)
```

Run: demo03_03_01

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_01.py
5050

Process finished with exit code 0
```

例如删除列表对象中所有偶数。

```
1  #删除列表对象中所有偶数。
2  x = list(range(20))        # 创建列表对象
3  for i in x:                # 从0循环到19
4      x.remove(i)            # 删除列表对象中下标为i的值
5  else:
6      print("delete over")   # for循环正常执行完成后，执行的else部分
7  print(x)
```

Run: demo03_03_02

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_02.py
delete over
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]

Process finished with exit code 0
```

for循环嵌套是指在for循环里有一个或多个for语句，循环里面再嵌套一重循环的叫双重循环，嵌套两层以上的叫多重循环。

例如使用两个for循环打印出九九乘法表，使用for循环和range()函数，变量i控制外层循环，变量j是控制内层循环的次数。

```
# 打印九九乘法表
for i in range(1, 10):            # 外循环循环9次
    for j in range(1, i + 1):     # 内循环控制每行输出的个数
        print(str(j) + "x" + str(i) + '=' + str(i * j), end=" ")      # 把数值类型转换成字符型然后进行输出
        # print('%dx%d=%2d'%(j, i, j*i), end=' ')                     # 使用转义字符进行输出
    print()     # 换行
```

for i in range(1, 10) › for j in range(1, i + 1)

Run: demo03_03_03

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_03.py
1x1=1
1x2=2 2x2=4
1x3=3 2x3=6 3x3=9
1x4=4 2x4=8 3x4=12 4x4=16
1x5=5 2x5=10 3x5=15 4x5=20 5x5=25
1x6=6 2x6=12 3x6=18 4x6=24 5x6=30 6x6=36
1x7=7 2x7=14 3x7=21 4x7=28 5x7=35 6x7=42 7x7=49
1x8=8 2x8=16 3x8=24 4x8=32 5x8=40 6x8=48 7x8=56 8x8=64
1x9=9 2x9=18 3x9=27 4x9=36 5x9=45 6x9=54 7x9=63 8x9=72 9x9=81

Process finished with exit code 0
```

例如求1！+2！+3！+4！+…+10！的和

```
1  # 求1! +2! +3! +4! +…+10! 的和
2  Sum = 0
3  for i in range(1, 11):   # 外循环完成累加
4      m = 1                # 阶乘初始化为1
5      for j in range(1, i + 1):   # 内循环完成i!的计算
6          m *= j           # m为i的阶乘
7      Sum += m             # 累加
8  print(Sum)
```

for i in range(1, 11)

Run: demo03_03_04

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_04.py
4037913

Process finished with exit code 0
```

break语句，它的作用是跳出循环或叫终止循环，执行循环后面的语句。continue语句是结束本次循环（循环体中continue后面的语句不执行），进入下一次循环。

例如循环条件为True，当i等于7的时候强制跳出循环。

```
1  # 循环条件为True，当i等于7的时候强制跳出循环。
2  i = 1
3  while True:          # 循环条件永远为True
4      if i == 7:       # 判断变量i的值是否等于7
5          break        # 如果变量i等于7就跳出循环
6      print(i, end=' ')
7      i += 1
8  else:
9      print('yes')
```

```
Run: demo03_03_05
D:\Python\Python36\python.exe F:/python/new/demo03_03_05.py
1 2 3 4 5 6
Process finished with exit code 0
```

例如把50~80的不能被3整除的数输出。

```
1  # 把50~80的不能被3整除的数输出
2  for i in range(50, 80):   # 从50循环到79
3      if i % 3 == 0:        # 如果能被3整除就不输出
4          continue
5      print(i, end=' ')
```

Run: demo03_03_06

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_06.py
50 52 53 55 56 58 59 61 62 64 65 67 68 70 71 73 74 76 77 79
Process finished with exit code 0
```

在循环体中可以包含另一个循环或分支语句，在分支语句中也可以包含另一个分支或循环。

例如将小写字母转换成大写字母，大写字母转换成小写字母，注意代码缩进。

```
1   # 将小写字母转换成大写字母，大写字母转换成小写字母
2   x = input("请输入字母：")
3   for i in range(len(x)):
4       if (x[i] >= 'a') and (x[i] <= 'z'):          # 判断是否为小写字母
5           print(chr(ord(x[i]) - 32), end='')       # 首先将字符转换成ASCII码进行计算，然后再将ASCII码转换成字符
6       elif (x[i] >= 'A') and (x[i] <= 'Z'):        # 判断是否为大写字母
7           print(chr(ord(x[i]) + 32), end='')
8       if x[i] == ' ':                              # 如果遇到空格，原样输出
9           print(end=" ")
10  else:
11      print('\nover!')
```

Run: demo03_03_07

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_07.py
请输入字母：Hello world
hELLO WORLD
over!

Process finished with exit code 0
```

# 3.3 循环流程控制



当不知道循环次数，但知道循环条件时，一般使用while语句。与for循环类似，可在循环体中使用break和continue语句。else部分可以省略。

注意：在Python中没有do...while语句。其语法结构如下。

```
while 循环条件:
    循环体（语句块1）
else:
    语句块2
```

例如使用双重while循环打印出一个倒三角形图案。

```
1  # 打印出一个倒三角形的图案
2  i = 0
3  while i < 5:                # 外循环0到4
4      j = 5                   # 每循环一次变量j初始化为5
5      while j > i:            # 内循环从5减到变量i
6          print('。', end=' ')
7          j -= 1              # 控制内循环
8      print()
9      i += 1                  # 控制外循环
```

Run: demo03_03_08

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_08.py
。 。 。 。 。
。 。 。 。
。 。 。
。 。
。
```

例如求50以内所有5的倍数的和。

```
1   # 求50以内所有5的倍数的和
2   i = 1
3   Sum = 0
4   while i <= 50:          # 从1循环到50
5       if i % 5 == 0:      # 判断变量i是否能被5整除
6           Sum += i
7           print(i, end=' ')
8       i += 1              # 循环控制变量
9   else:                   # 循环正常结束，就执行else部分
10      print("\nover")
11  print(Sum)
```

Run: demo03_03_09

```
D:\Python\Python36\python.exe F:/python/new/demo03_03_09.py
5 10 15 20 25 30 35 40 45 50
over
275
```

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.5 小结

习题

# 3.4 实验



1、从键盘输入三个同学的成绩，然后找出最高分。

# 3.4 实验



2、输入三个同学的成绩，然后大到小排列。

# 3.4 实验



1、求出1000以内的所有完数，如6=1+2+3除了它自身以外的因子之和等于它本身叫完数。

# 3.4 实验



2、用循环语句求1+22+333+4444+55555的和。

# 3.4 实验



1、求出2000-2100的所有闰年，条件是能被4整除且不能被100整除，或者能被400整除的是闰年。

# 3.4 实验



2、输入两个正整数，并求出它们的最大公约数和最小公倍数。

# 3.4 实验



1、输出100以内的所有质数。

# 3.4 实验



2、求100以内最大的10个质数的和。

# 3.4 实验



1、求1-10的所有偶数的和。

# 3.4 实验



2、将10-20不能被2或3整除的数输出。

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.4 小结

习题

# 3.5 小结



本章讲解了Python的流程控制：if分支、for循环和while循环。if、for和while语法上很简单，但通过组合或嵌套，可以实现各种简单到复杂的程序逻辑结构。

为了保证程序流程控制的灵活性，Python提供了continue和break两个语句来控制循环语句。continue语句用来结束本次循环，提前进入下一次循环。break语句用于强制退出循环，不执行循环体中剩余的循环次数。

# 第三章 流程控制

3.1 条件语句

3.2 条件流程控制

3.3 循环流程控制

3.4 实验

3.5 小结

习题

# 习题：

1. _____语句是else和if的组合。
2. _____、_____不能单独和if分支配合使用。
3. 每个流程结构语句后面必须要有_____。
4. Python中的流程控制语句有_____、_____和_____。
5. 当循环_____结束时才会执行else部分。

# 感谢聆听